

GIAV

Geovisor Inteligente de Accidentabilidad Vial

Manual Técnico

Documentación técnica de arquitectura, base de datos y código fuente del sistema

Proyecto	GIAV – Geovisor Inteligente de Accidentabilidad Vial
Versión del sistema	1.0
Autor(es)	Oscar Eduardo Belalcazar Gabriel Angel Chaña Julio Sebastian Preciado Jan Carlos Ararat
Fecha	27/06/2026

Tabla de Contenido

Tabla de Contenido	2
1. Alcance	3
2. Descripción de los Procesos	3
2.1 Actores del sistema	3
3. Diagramas y Arquitectura.....	3
3.1 Patrón arquitectónico	3
3.2 Flujo de una petición	4
3.3 Diagrama de flujo MVC	4
3.4 Componente de visualización geoespacial.....	4
4. Administración de Usuarios y Roles	5
4.1 Carga de permisos en sesión.....	6
4.2 Verificación de permisos	6
4.3 Gestión de roles desde la interfaz.....	6
5. Modelo Relacional de la Base de Datos	6
5.1 Núcleo de acceso y seguridad	6
5.2 Reportes y solicitudes	7
5.3 Catálogos de apoyo	7
5.4 Trazabilidad	7
6. Descripción de la Plataforma	7
7. Documentación del Código Fuente.....	8
7.1 Estructura de directorios.....	8
7.2 Relación de módulos y controladores.....	8
7.3 Convención del enrutamiento.....	9
8. Descripción de los Acuerdos de Niveles de Servicio (ANS)	9

1. Alcance

Este manual va dirigido al área de soporte técnico de la organización o a quien haga sus veces. Describe la arquitectura, los componentes y la estructura del sistema GIAV, permitiendo entender su funcionamiento global y posibilitar su mantenimiento.

Se cubren los siguientes aspectos: descripción de procesos, diagramas de arquitectura, administración de usuarios y roles, modelo relacional de la base de datos, descripción de la plataforma tecnológica y documentación del código fuente. No se incluyen instrucciones de uso funcional, las cuales se encuentran en el Manual de Usuario, ni el procedimiento de instalación del servidor, documentado en el Manual de Instalación.

2. Descripción de los Procesos

El sistema GIAV apoya el proceso de gestión de la información de accidentalidad vial del municipio de Cali, cubriendo los siguientes subprocesos:

- Gestión de acceso: registro, autenticación y recuperación de contraseña de usuarios.
- Gestión de reportes: registro de accidentes y solicitudes relacionadas con señalización vial y reductores de velocidad.
- Gestión de historial: consulta y edición de los reportes registrados, según el rol del usuario.
- Gestión de usuarios y roles: administración de cuentas, asignación de roles y configuración de permisos por módulo.
- Visualización geoespacial: representación cartográfica de la malla vial, división político-administrativa y reportes de accidentalidad mediante un servidor de mapas.
- Gestión de PQRSF y estadísticas: registro de peticiones, quejas, reclamos, sugerencias y felicitaciones, y generación de indicadores de accidentalidad.

2.1 Actores del sistema

Rol	Descripción
Ciudadano	Usuario externo que reporta accidentes y solicita intervenciones de señalización vial. No tiene acceso a módulos administrativos.
Funcionario	Usuario interno con acceso al historial de reportes, manuales, PQRSF y estadísticas. No gestiona usuarios ni reportes de accidentes directamente.
Administrador del sistema	Usuario con acceso total a todos los módulos, incluyendo la gestión de usuarios, roles y permisos.

3. Diagramas y Arquitectura

3.1 Patrón arquitectónico

El sistema GIAV está desarrollado siguiendo el patrón Modelo – Vista – Controlador (MVC), implementado en PHP procedural orientado a objetos, sin el uso de un framework. La estructura de carpetas refleja directamente este patrón:

Carpeta	Responsabilidad
controller/	Contiene un subdirectorio por módulo de negocio (Acceso, Usuario, Reportes, Historial, Mapa, Estadísticas, PQRSF, Manual). Cada controlador recibe la petición HTTP, ejecuta la lógica de negocio y construye las consultas SQL.
model/	Contiene una clase por módulo que extiende de MasterModel. En la mayoría de los casos no implementa métodos propios: la lógica SQL se ejecuta directamente desde el controlador usando los métodos heredados (select, insert, update, delete).
view/	Contiene los archivos PHP con el marcado HTML de cada pantalla, organizados por módulo, y la subcarpeta partials/ con los fragmentos comunes (cabecera, menú, barra de navegación, pie de página, mapa).
web/	Punto de entrada público de la aplicación. Contiene index.php (panel autenticado), ajax.php (resolutor de peticiones AJAX/POST), login.php, registro.php y los recursos estáticos (assets/).
lib/	Librerías y utilidades transversales: helpers.php (funciones de enrutamiento), Permisos.php (verificación de permisos por rol), conf/ (parámetros de conexión a base de datos) y las librerías de terceros PHPMailer y PHPEXcel.

3.2 Flujo de una petición

Toda petición al sistema autenticado sigue el siguiente recorrido:

1. El navegador solicita web/index.php, opcionalmente con los parámetros modulo, controlador y funcion en la URL.
2. index.php valida que exista una sesión activa (\\$_SESSION['auth']); de lo contrario redirige a login.php.
3. Si la petición incluye un módulo sujeto a control de permisos, se invoca Permisos::hasModule() antes de continuar.
4. La función resolver(), definida en lib/helpers.php, construye dinámicamente el nombre de la clase controladora y el archivo a incluir, a partir de los parámetros de la URL.
5. El controlador ejecuta la consulta SQL correspondiente a través del modelo (heredado de MasterModel) e incluye la vista PHP que renderiza el resultado.

Las peticiones que se originan desde formularios HTML (creación y edición de reportes, login, registro) se envían a web/ajax.php, el cual replica la misma lógica de resolver() pero sin el layout completo del panel administrativo, permitiendo respuestas livianas o redirecciones.

3.3 Diagrama de flujo MVC

- | | |
|-----------------------|---|
| 1. Navegador | -> GET/POST a web/index.php (con parametros modulo, controlador, funcion) |
| 2. web/index.php | -> valida sesion y permisos, luego llama a resolver() |
| 3. resolver() | -> incluye e instancia {Modulo}Controller |
| 4. {Modulo}Controller | -> ejecuta SQL via {Modulo}Model (MasterModel) sobre PostgreSQL |
| 5. {Modulo}Controller | -> incluye la vista view/{Modulo}/{Vista}.php |
| 6. Vista PHP | -> renderiza el HTML final que recibe el navegador |

3.4 Componente de visualización geoespacial

El módulo de mapas no utiliza una librería JavaScript de mapas convencional (como Leaflet o Google Maps), sino MapServer, un servidor de mapas que procesa archivos Mapfile (.map) y datos vectoriales (Shapefiles y capas PostGIS) para generar imágenes ráster servidas a través de un programa CGI (mapserv.exe).

Componente	Función
web/cali.map	Mapfile que define las capas del mapa (Cali, Comunas, Barrio, MallaVial) a partir de Shapefiles estáticos, y la capa Accidentes, que consulta en vivo la tabla reporte_accidente mediante una conexión PostGIS.
web/assets/shapeCali41/	Shapefiles (.shp, .dbf, .shx, .prj) con la cartografía base de Cali: área municipal, comunas, barrios y malla vial.
cgi-bin/mapserv.exe	Ejecutable CGI de MapServer (distribuido en MS4W), responsable de renderizar las capas del Mapfile como imagen y devolverla al navegador.
web/misc/lib/mscross-1.1.9.js	Librería JavaScript cliente (MapScript Cross-browser) que construye las peticiones al CGI, gestiona el paneo, zoom y la selección de coordenadas mediante clic sobre el mapa.
view/partials/contenido.php	Vista que inicializa el visor principal del mapa (mostrado en la pantalla de inicio del panel) y construye dinámicamente la lista de capas activas según los checkboxes marcados por el usuario.

La capa Accidentes del Mapfile se conecta directamente a PostgreSQL mediante CONNECTIONTYPE postgres, seleccionando únicamente los registros de reporte_accidente cuya columna coordenadas no es nula. Las coordenadas se almacenan en formato GEOMETRY(POINT, 4326) (WGS84).

4. Administración de Usuarios y Roles

El sistema implementa un modelo de control de acceso basado en roles (RBAC), compuesto por las siguientes entidades:

Tabla	Descripción
usuarios	Almacena los datos personales del usuario, sus credenciales y la referencia a su rol (id_rol) y tipo de documento (id_tipo_doc).
roles	Catálogo de roles del sistema (Ciudadano, Administrador Sistema, Funcionario, y cualquier rol adicional creado por un Administrador).
modulos	Catálogo de módulos funcionales del sistema (Mapa, Reportes, Historial Reportes, Manual, PQRSF, Estadísticas, Gestión de Usuarios).
acciones	Catálogo de acciones posibles sobre un módulo (VER, CREAR, EDITAR, ELIMINAR).
permisos	Combinación específica de un módulo (id_modulo) y una acción (accion), con un nombre descriptivo.
rol_permiso	Tabla de relación N:M entre roles y permisos. Cada fila indica que un rol específico tiene habilitado un permiso específico.
token	Almacena los códigos de verificación de 6 dígitos generados durante el proceso de recuperación de contraseña, junto con su fecha de creación e intentos fallidos.

4.1 Carga de permisos en sesión

Al iniciar sesión correctamente (`AccesoController::login()`), el sistema consulta todos los permisos asociados al rol del usuario mediante un JOIN entre `rol_permiso`, `permisos` y `modulos`, y los almacena en `\$_SESSION['permisos']` como un arreglo asociativo. Esta carga ocurre una sola vez por sesión.

4.2 Verificación de permisos

La clase `lib/Permisos.php` expone dos métodos estáticos utilizados a lo largo de todo el sistema:

- `Permisos::hasModule(idModulo)`: retorna verdadero si el rol del usuario en sesión tiene al menos un permiso asociado a ese módulo. Se utiliza en `web/index.php` para bloquear el acceso completo a un módulo.
- `Permisos::hasPermission(idModulo, idAccion)`: retorna verdadero si el rol tiene el permiso específico de esa combinación módulo + acción. Se utiliza en las vistas para mostrar u ocultar botones de acción (por ejemplo, el botón Editar en las tablas del módulo Historial).

La verificación de permisos en las vistas (ocultar botones) es únicamente una medida de experiencia de usuario. La protección real contra acceso no autorizado debe reforzarse a nivel de controlador en cada módulo sensible, replicando la verificación de `Permisos::hasModule()` o `Permisos::hasPermission()` al inicio de cada método público.

4.3 Gestión de roles desde la interfaz

El módulo `Usuario/GestionRolesController` permite, exclusivamente al rol Administrador:

- Listar los roles existentes (`getList`).
- Crear un rol nuevo asignando un conjunto de permisos mediante checkboxes (`getCreate` / `postCreate`).
- Editar los permisos de un rol existente, agregando o quitando filas en `rol_permiso` según los checkboxes marcados (`getEdit` / `postUpdate`).

5. Modelo Relacional de la Base de Datos

El sistema utiliza PostgreSQL como motor de base de datos, con la extensión PostGIS habilitada para el manejo de datos geoespaciales. El esquema se compone de los siguientes grupos de tablas:

5.1 Núcleo de acceso y seguridad

Tabla	Columna Principal
usuarios	id (PK), id_tipo_doc (FK), numero_id, nombre, apellido, telefono, correo, contrasena, direccion, id_rol (FK), id_estado (FK)
roles	id_rol (PK), nombre_rol
modulos	id_modulo (PK), nombre, activo
acciones	id_accion (PK), nombre, descripcion
permisos	id_permiso (PK), nombre, accion (FK a acciones), activo, id_modulo (FK)

Tabla	Columna Principal
rol_permiso	id_rol_permiso (PK), id_rol (FK), id_permiso (FK)
token	id_token (PK), id_usuario (FK), codigo, usado, intentos, fecha_creacion

5.2 Reportes y solicitudes

Tabla	Propósito
reporte_accidente	Reportes de accidentes de tránsito, incluyendo la columna coordenadas de tipo GEOMETRY(POINT,4326) para la ubicación geográfica.
sol_nueva_senal	Solicitudes de instalación de una nueva señal de tránsito.
sol_senal_mal_estado	Solicitudes de reparación de una señal existente en mal estado.
sol_nuevo_reductor	Solicitudes de instalación de un nuevo reductor de velocidad.
sol_reductor_mal_estado	Solicitudes de reparación de un reductor existente en mal estado.
sol_via_mal_estado	Solicitudes relacionadas con el mal estado de una vía.
pqrsf	Peticiones, quejas, reclamos, sugerencias y felicitaciones registradas por los usuarios.

5.3 Catálogos de apoyo

El sistema cuenta además con tablas de catálogo utilizadas como listas de selección en los formularios: tipo_documento, categoria_senal, categoria_reductor, tipo_choque, tipo_vehiculo, tipo_dano_senal, tipo_dano_via, tipo_dano_reductor, tipo_pqrsf, estado, orientacion_senal, tipo_senal y tipo_reductor.

5.4 Trazabilidad

Las tablas auditoria_usuarios y auditoria_solicitudes registran el historial de cambios sobre usuarios y solicitudes respectivamente, con fines de trazabilidad administrativa.

6. Descripción de la Plataforma

GIAV es una aplicación web desarrollada sobre una variante de la pila LAMP (Linux/Windows, Apache, PostgreSQL, PHP), con el siguiente detalle de herramientas:

Componente	Tecnología/Versión
Lenguaje de programación del servidor	PHP 5.2
Servidor web	Apache (distribuido junto con MS4W – MapServer for Windows)
Sistema gestor de base de datos	PostgreSQL con extensión PostGIS
Servidor de mapas	MapServer (MS4W), vía CGI mapserv.exe
Librería de mapas en el cliente	mscross 1.1.9 (MapScript Cross-browser)
Envío de correo electrónico	PHPMailer 5.2.28, mediante SMTP sobre Gmail
Exportación de hojas de cálculo	PHPExcel

Componente	Tecnología/Versión
Interfaz gráfica / Theme administrativo	Kaiadmin (HTML5, CSS3, JavaScript, jQuery, Bootstrap)
Encriptación de contraseñas	MD5 con salt fijo de aplicación

Debido a que el proyecto utiliza PHP 5.2, no están disponibles funciones introducidas en versiones posteriores, tales como `password_hash()`, `password_verify()`, `random_bytes()`, `random_int()`, el operador de coalescencia nula (`??`) ni la sintaxis de arreglos con corchetes (`[]`). El código debe escribirse utilizando `array()`, `isset()` con operador ternario, y `mt_rand()` como alternativas compatibles.

7. Documentación del Código Fuente

Este capítulo describe la estructura de carpetas del proyecto y la convención de nomenclatura utilizada para los controladores, modelos y vistas, con el fin de facilitar el mantenimiento del código.

7.1 Estructura de directorios

Directorio	Descripción
controller/{Modulo}/	Clases controladoras. Convención de nombre: {NombreModulo}Controller.php
model/{Modulo}/	Clases modelo, todas heredan de MasterModel. Convención: {NombreModulo}Model.php
view/{Modulo}/	Vistas PHP asociadas a cada controlador.
view/partials/	Fragmentos compartidos: head.php, menu.php, navbar.php, footer.php, script.php, contenido.php (mapa principal).
web/	index.php (panel autenticado), ajax.php (resolutor AJAX/POST), login.php, registro.php, recuperación de contraseña, cali.map y recursos estáticos en assets/.
lib/helpers.php	Define <code>getUrl()</code> , <code>resolver()</code> y <code>redirect()</code> , utilizadas en todo el proyecto para construir URLs y despachar peticiones.
lib/Permisos.php	Clase estática de verificación de permisos, descrita en el capítulo 4.
lib/conf/conf.php	Parámetros de conexión a la base de datos (servidor, usuario, contraseña, nombre de base, puerto).
lib/conf/connection.php	Apertura de la conexión PostgreSQL utilizando los parámetros de conf.php.
model/MasterModel.php	Clase base de la que heredan todos los modelos; expone los métodos <code>select()</code> , <code>insert()</code> , <code>update()</code> y <code>delete()</code> sobre <code>pg_query()</code> .

7.2 Relación de módulos y controladores

Modulo	Controlador	Propósito
Acceso	AccesoController	Inicio de sesión, recuperación de contraseña (envío y validación de código, actualización de clave).

Modulo	Controlador	Propósito
Usuario	UsuarioController, GestionUsuarioController, GestionRolesController	Registro público, perfil de usuario, listado de usuarios, gestión de roles y permisos.
Reportes	ReportesAController, ReportesNSController, ReportesSMEController, ReportesSolicitudNRController, ReportesRMEController, SolicitudVMEController	Registro de accidentes y de las cinco solicitudes relacionadas con señalización y reductores.
Historial	MiHistorialController	Consulta y filtrado de los reportes registrados, con acceso a edición según permisos.
Mapa	mapaController	Despliegue del visor cartográfico principal.
Estadísticas	ZonaMayAccidentabilidadController	Cálculo y presentación de indicadores de accidentalidad.
PQRSF	PqrsfController	Registro y consulta de peticiones, quejas, reclamos, sugerencias y felicitaciones.
Manual	ManualUController, ManualSController	Despliegue del manual de usuario y del manual de señalización vial.

7.3 Convención del enrutamiento

El sistema no utiliza rutas amigables (friendly URLs). Todas las peticiones siguen el patrón:

```
{pagina}.php?modulo={Modulo}&controlador={Controlador}&funcion={metodo}
```

La función getUrl(modulo, controlador, funcion, parametros, pagina) (lib/helpers.php) construye esta cadena de manera centralizada, evitando que las vistas concatenen URLs manualmente. El parámetro pagina por defecto es index, y puede sobrescribirse a ajax para peticiones POST desde formularios.

8. Descripción de los Acuerdos de Niveles de Servicio (ANS)

Los usuarios técnicos que presten soporte de primer nivel sobre el sistema GIAV deben tener en cuenta los siguientes lineamientos generales de atención:

Nivel	Tipo de Incidente	Tiempo de respuesta
1	Bloqueo de acceso, restablecimiento de contraseña, errores de formularios	4 horas hábiles
2	Fallos del visor de mapas, errores de generación de reportes, problemas de permisos	1 día hábil
3	Fallos de base de datos, caída del servicio MapServer, errores estructurales	Según disponibilidad de un desarrollador, máximo 3 días hábiles

Los incidentes que no se puedan resolver en el nivel correspondiente deben escalar al siguiente nivel junto con la documentación del problema (descripción, mensaje de error exacto y pasos para reproducirlo).